



FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES

Departamento: Departamento de ciencias de la computación.

Carrera: Ingeniería en Software Asignatura:
Análisis y diseño de software NRC: 30724

Apellidos y nombres del estudiante: Díaz
Stiven, Leitón José, Manosalvas Gabriel.

ANTECEDENTES

1. El desarrollo de sistemas orientados a objetos modernos exige la adopción de esquemas arquitectónicos que mitiguen el acoplamiento y potencien la cohesión interna de los componentes. La arquitectura de tres capas se presenta como el estándar fundamental de segregación de responsabilidades, dividiendo el ecosistema de software en componentes de interacción con el usuario (Presentación), motores de orquestación de reglas de negocio (Lógica) y abstracciones de almacenamiento de datos (Persistencia).
2. Conforme las aplicaciones incrementan su complejidad funcional, la rigidez inherente a las estructuras primitivas genera problemas de escalabilidad. Para solventar este fenómeno sin degradar la arquitectura base, se recurre a los patrones de diseño orientados a objetos de GoF (Gang of Four). Los patrones estructurales (como *Adapter* y *Decorator*) permiten armonizar interfaces incompatibles y acoplar responsabilidades transversales dinámicamente. Por otro lado, los patrones comportamentales (como *Observer* y *Strategy*) optimizan la comunicación inter-objetos y la flexibilidad algorítmica ante requerimientos cambiantes en el tiempo de ejecución.
3. El caso de estudio planteado consiste en un sistema de mantenimiento de datos para la entidad Estudiante, la cual se encuentra estrictamente tipificada por los atributos esenciales ID, Nombres y Edad. A través de este dominio comprensible, se evidencia la trazabilidad metodológica completa desde los requisitos del sistema hasta la verificación técnica y de pruebas automatizadas.

OBJETIVO GENERAL

Diseñar e implementar una solución integral de software orientada a objetos para el ecosistema CRUD del Estudiante, fundamentada en una arquitectura de tres capas e integrada de forma secuencial con patrones de diseño estructurales (*Adapter*, *Decorator*) y de comportamiento (*Observer*, *Strategy*), con la finalidad de maximizar los atributos de calidad de mantenibilidad, extensibilidad y bajo acoplamiento del sistema, garantizando una trazabilidad metodológica estricta con la Especificación de Requisitos de Software (ERS/SRS).

OBJETIVOS ESPECÍFICOS

1. Implementar la arquitectura base dividida en tres capas físicas y lógicas disjuntas (Presentación, Lógica de Negocio y Datos), asegurando que los componentes de

la interfaz de usuario carezcan de conexiones directas con los mecanismos de persistencia.

2. Construir un componente de adaptación (*Adapter*) para procesar flujos de datos externos con nomenclaturas disímiles a la entidad nativa, unificándolos bajo la estructura formal de Estudiante.
3. Aplicar el patrón *Decorator* para inyectar responsabilidades adicionales de auditoría y validación temporal a los flujos transaccionales del CRUD, blindando el código del servicio de negocio original de modificaciones directas.
4. Configurar un motor de eventos mediante el patrón *Observer*, de tal manera que las operaciones transaccionales del CRUD desencadenan notificaciones en tiempo real hacia observadores heterogéneos.
5. Integrar el patrón *Strategy* en la lógica operacional para dotar al sistema de la capacidad de alterar dinámicamente entre múltiples criterios algorítmicos de ordenamiento y búsqueda de información.

DESARROLLO

PARTE 1: U2T1. Diseño e implementación de la arquitectura de tres capas para el CRUD del estudiante

Descripción del problema

El sistema debe permitir la gestión básica de estudiantes. Si la interfaz gráfica concentrará toda la lógica, el código sería difícil de mantener, ya que tendría que validar datos, crear objetos, guardar registros, actualizar la tabla y eliminar información en un mismo lugar. Del mismo modo, si la vista accediera directamente a la lista de estudiantes, se rompería la separación de responsabilidades solicitada en la guía.

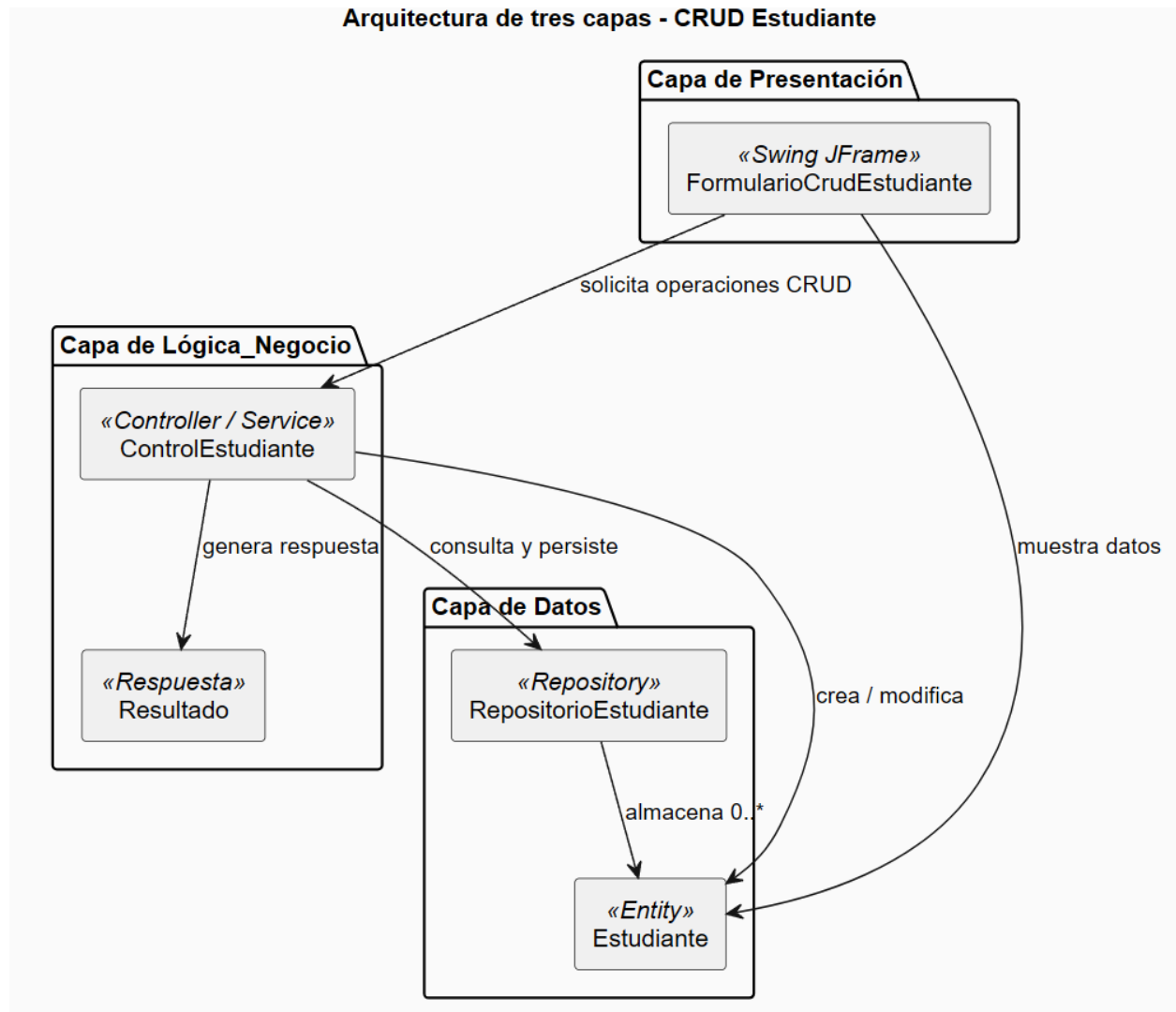
La solución consiste en organizar el CRUD en tres capas. La capa de Presentación queda representada por `FormularioCrudEstudiante`, la capa de Lógica_Negocio por `ControlEstudiante` y `Resultado`, y la capa de Datos por `RepositorioEstudiante`. La clase `Estudiante` funciona como entidad de dominio compartida por las capas.

Elemento de la arquitectura y clases del proyecto

Capa / elemento	Clase en el proyecto	Responsabilidad
Presentación	<code>FormularioCrudEstudiante</code>	Construye la interfaz Swing, recibe los datos del usuario, ejecuta acciones de botones y muestra mensajes o resultados en la tabla.
Lógica_Negocio	<code>ControlEstudiante</code>	Valida datos, coordina las operaciones CRUD y evita que la vista acceda directamente al repositorio.
Lógica_Negocio	<code>Resultado</code>	Encapsula el estado de cada operación mediante un indicador de éxito y un mensaje para la interfaz.

Datos	RepositorioEstudiante	Administra la lista en memoria, permite guardar, buscar, actualizar, eliminar y listar estudiantes.
Modelo / entidad	Estudiante	Representa la entidad principal del sistema con los atributos id, nombre y edad.

Modelo de arquitectura en PlantUML



```

@startuml
skinparam componentStyle rectangle
skinparam shadowing false
title Arquitectura de tres capas - CRUD Estudiante

package "Capa de Presentación" {
  [FormularioCrudEstudiante] <<Swing JFrame>>
}

package "Capa de Lógica_Negocio" {
  [ControlEstudiante] <<Controller / Service>>
  [Resultado] <<Respuesta>>
}

package "Capa de Datos" {
  [RepositorioEstudiante] <<Repository>>
  [Estudiante] <<Entity>>
}

' ——— RELACIONES

[FormularioCrudEstudiante] --> [ControlEstudiante] : solicita operaciones CRUD
[ControlEstudiante] --> [Resultado] : genera respuesta
[ControlEstudiante] --> [RepositorioEstudiante] : consulta y persiste
[ControlEstudiante] --> [Estudiante] : crea / modifica
[RepositorioEstudiante] --> [Estudiante] : almacena 0..*
[FormularioCrudEstudiante] --> [Estudiante] : muestra datos

@enduml

```

El ControlEstudiante funge como único intermediario, procesando las solicitudes y estandarizando las respuestas a través del objeto Resultado. Esto permite que el RepositorioEstudiante opere de forma independiente, dedicándose exclusivamente a la persistencia de la entidad Estudiante sin que la vista conozca los detalles de la base de datos.

Matriz de trazabilidad entre requisitos, componentes y evidencias

ID	Requisito funcional	Operación	Componente principal	Evidencia requerida
RF-01	El sistema debe permitir registrar un estudiante con ID, Nombres y Edad.	Crear	FormularioCrudEstudiante, ControlEstudiante, RepositorioEstudiante, Estudiante	Captura del formulario con datos ingresados y mensaje de registro exitoso.

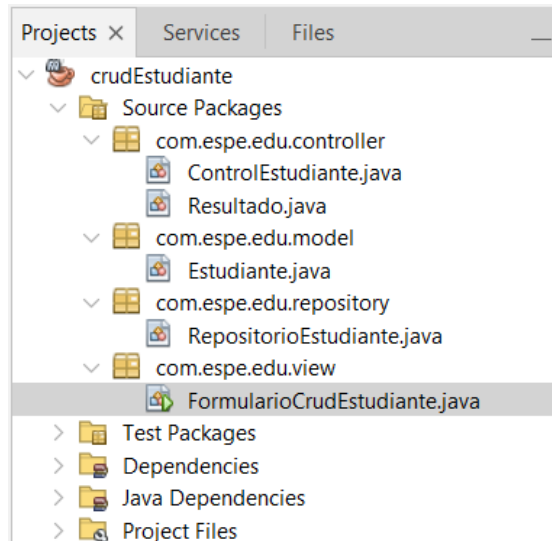
RF-02	El sistema debe permitir consultar/listar los estudiantes registrados.	Consultar	FormularioCrudEstudiante, ControlEstudiante, RepositorioEstudiante	Captura de la tabla con uno o más estudiantes visibles.
RF-03	El sistema debe permitir actualizar los datos de un estudiante existente.	Actualizar	FormularioCrudEstudiante, ControlEstudiante, RepositorioEstudiante	Captura antes/después con cambio de nombre o edad y mensaje de actualización.
RF-04	El sistema debe permitir eliminar un estudiante por ID.	Eliminar	FormularioCrudEstudiante, ControlEstudiante, RepositorioEstudiante	Captura de confirmación de eliminación y tabla sin el registro eliminado.
RNF-01	El sistema debe mantener separación entre presentación, lógica y datos.	Arquitectura	Paquetes view, controller, repository y model	Captura de la estructura de paquetes del proyecto.

Validaciones y restricciones del sistema

Restricción / validación	Ubicación	Descripción
ID obligatorio	ControlEstudiante.validarDatos()	No se acepta un ID nulo o vacío.
Nombre obligatorio	ControlEstudiante.validarDatos()	No se acepta un nombre nulo o vacío.
Edad mayor a cero	ControlEstudiante.validarDatos() y FormularioCrudEstudiante	La edad debe ser numérica y mayor que cero.
ID único	RepositorioEstudiante.existsId()	Antes de guardar se verifica que no exista otro estudiante con el mismo ID.
Persistencia en memoria	RepositorioEstudiante	Los datos se almacenan en una lista durante la ejecución del programa; no existe base de datos externa.

Evidencias de ejecución

Estructura del proyecto CRUD - 3 capas



Ejecución del CRUD - 3 capas

The screenshot shows the 'CRUD Estudiantes' application window. It features a form with three input fields for 'ID:', 'Nombre:', and 'Edad:'. Below the form are four buttons: 'Agregar', 'Actualizar', 'Eliminar', and 'Mostrar Todo'. At the bottom, there is a table with three columns: 'ID', 'Nombre', and 'Edad'. The table is currently empty.

ID	Nombre	Edad
----	--------	------

CRUD Estudiantes

ID:

Nombre:

Edad:

ID	Nombre	Edad
1234	Ariel Gutierrez	23
22211111220	Jorge	24
22211111221	Ivan	23
22211111222	Miguel	30

CRUD Estudiantes

ID:

Nombre:

Edad:

ID	Nombre	Edad
1234	Ariel Gutierrez	23
22211111220	Jorge	24
22211111221	Ivan	23
22211111222	Miguel	30

Message

Estudiante actualizado

CRUD Estudiantes

ID:

Nombre:

Edad:

ID	Nombre	Edad
1234	Ariel Gutierrez	22
22211111220	Jorge	24
22211111221	Ivan	23
22211111222	Miguel	30
22211111220	Jorge	24
22211111221	Ivan	23
22211111222	Miguel	30

Message

Estudiante eliminado

CONCLUSIONES

- La arquitectura de tres capas permite organizar el CRUD de estudiantes de forma clara, separando la interfaz gráfica, las reglas de negocio y el almacenamiento en memoria.
- El código entregado cumple de forma general con la separación solicitada, ya que `FormularioCrudEstudiante` delega las operaciones a `ControlEstudiante` y no manipula directamente la lista de estudiantes.
- La matriz de trazabilidad permite vincular los requisitos funcionales mínimos del CRUD con las clases implementadas y con las evidencias que deben presentarse para demostrar su funcionamiento.
- La solución queda preparada para las siguientes tareas de la guía, donde se podrán incorporar patrones estructurales y comportamentales sobre la misma base arquitectónica.

RECOMENDACIONES

- Mantener la capa de Presentación libre de lógica de persistencia y evitar que la vista acceda directamente a `RepositorioEstudiante`.
- Agregar una clase `ServicioEstudiante` en futuras iteraciones para reforzar la separación entre controlador de interfaz y lógica de negocio.
- Incorporar persistencia en archivos o base de datos si se requiere conservar los registros después de cerrar la aplicación.
- Adjuntar capturas reales de ejecución y, si es posible, pruebas unitarias para fortalecer la evidencia del funcionamiento base.

PARTE2: U2T2. Aplicación de Adapter y Decorator en el CRUD del estudiante

Descripción del problema

Con la arquitectura de tres capas base de la U2T1 implementada, el sistema enfrenta dos nuevos desafíos de diseño que amenazan con degradar la separación de responsabilidades si se resuelven de forma ingenua:

Entradas Incompatibles (Problema de Integración): El sistema requiere procesar listados de estudiantes desde fuentes externas. Un archivo CSV utiliza los campos `código`, `nombreCompleto` y `anios`, mientras que un archivo JSON externo provee los campos en inglés `studentId`, `name` y `age`. Si el controlador o la vista intentan interpretar estas nomenclaturas directamente, se acoplarían fuertemente a formatos externos cambiantes.

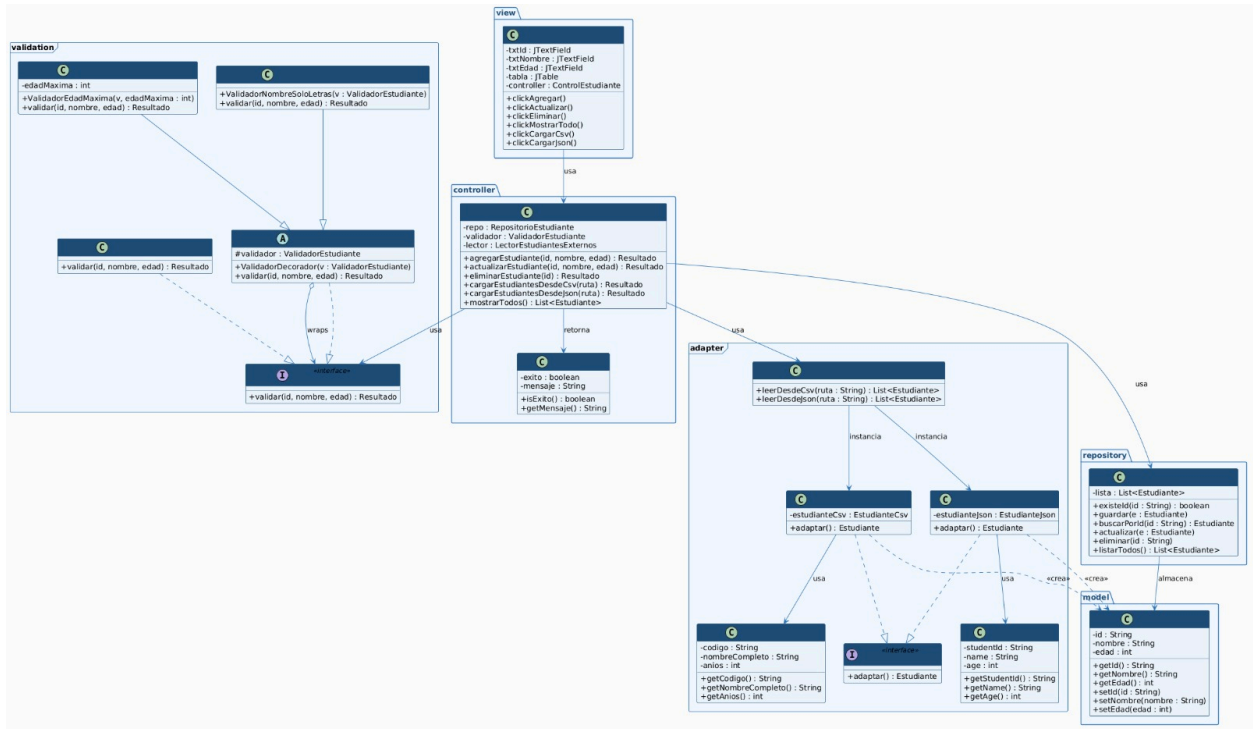
Comportamientos Transversales Cambiantes (Problema de Extensión): El sistema base implementa una validación elemental (campos obligatorios). El negocio exige agregar de forma dinámica y paramétrica nuevas reglas de validación (por ejemplo, validar que la edad no supere un rango lógico máximo y que el nombre no contenga números), modificar la clase de validación original violaría el principio de Abierto/Cerrado (OCP) y aumentaría el riesgo de regresión sobre el CRUD básico.

La solución consiste en aplicar dos patrones estructurales de GoF, el patrón Adapter unifica los flujos de datos externos disímiles bajo una interfaz común, traduciendo las propiedades externas al modelo nativo Estudiante, el patrón Decorator permite envolver la validación básica con filtros adicionales apilables en tiempo de ejecución mediante composición, protegiendo las clases originales de modificaciones directas.

Elemento de la arquitectura y clases del proyecto

Capa / elemento	Clase en el proyecto	Responsabilidad
Integración adapter	AdaptadorEstudiante	Interfaz que define el contrato de adaptación unificado mediante el método adaptar().
Integración adapter	EstudianteCsv / EstudianteJson	Clases de transferencia de datos (DTO) que mapean las nomenclaturas nativas de los archivos externos.
Integración adapter	AdaptadorCsvEstudiante	Clase concreta que encapsula un objeto EstudianteCsv y traduce sus campos (codigo $\rightarrow$ id, nombreCompleto $\rightarrow$ nombre, anios $\rightarrow$ edad).
Integración adapter	AdaptadorJsonEstudiante	Clase concreta que encapsula un objeto EstudianteJson y traduce sus campos en inglés hacia las propiedades nativas del dominio.

Modelo de arquitectura en PlantUML



@startuml U2T2_Diagrama_Clases

skinparam classAttributeIconSize 0
skinparam backgroundColor #FFFFFF
skinparam defaultFontName Arial
skinparam defaultFontSize 11

```
skinparam class {  
    BackgroundColor #1F3864  
    BorderColor #1F3864  
    FontColor #FFFFFF  
    HeaderBackgroundColor #1F3864  
    HeaderFontColor #FFFFFF  
    AttributeFontColor #FFFFFF  
}
```

```
skinparam interface {  
    BackgroundColor #C8860A  
    BorderColor #C8860A  
    FontColor #FFFFFF  
    HeaderBackgroundColor #C8860A  
    HeaderFontColor #FFFFFF  
}
```

```
skinparam arrow {  
    Color #2E75B6  
    FontColor #1F3864  
    FontSize 10  
}
```

```
skinparam package {  
    BackgroundColor #EBF3FB  
    BorderColor #2E75B6  
    FontColor #1F3864  
    BorderThickness 1.5  
}
```

```
' _____  
' VIEW  
' _____
```

```
package "view" {  
    class FormularioCrudEstudiante {  
        - txtId : JTextField  
        - txtNombre : JTextField  
        - txtEdad : JTextField  
        - tabla : JTable  
        - controller : ControlEstudiante  
    }  
}
```

```

+ clickAgregar()
+ clickActualizar()
+ clickEliminar()
+ clickMostrarTodo()
+ clickCargarCsv()
+ clickCargarJson()
}
}

'
=====
' CONTROLLER
'
=====

package "controller" {
  class ControlEstudiante {
    - repo : RepositorioEstudiante
    - validador : ValidadorEstudiante
    - lector : LectorEstudiantesExternos
    --
    + agregarEstudiante(id, nombre, edad) : Resultado
    + actualizarEstudiante(id, nombre, edad) : Resultado
    + eliminarEstudiante(id) : Resultado
    + cargarEstudiantesDesdeCsv(ruta) : Resultado
    + cargarEstudiantesDesdeJson(ruta) : Resultado
    + mostrarTodos() : List<Estudiante>
  }

  class Resultado {
    - exito : boolean
    - mensaje : String
    --
    + isExito() : boolean
    + getMensaje() : String
  }
}

'
=====
' VALIDATION (Decorator)
'
=====

package "validation" {
  interface ValidadorEstudiante <<interface>> {
    + validar(id, nombre, edad) : Resultado
  }

  abstract class ValidadorDecorador {
    # validador : ValidadorEstudiante
    --
    + ValidadorDecorador(v : ValidadorEstudiante)
    + validar(id, nombre, edad) : Resultado
  }
}

```

```

}

class ValidadorBasico {
+ validar(id, nombre, edad) : Resultado
}

class ValidadorEdadMaxima {
- edadMaxima : int
--
+ ValidadorEdadMaxima(v, edadMaxima : int)
+ validar(id, nombre, edad) : Resultado
}

class ValidadorNombreSoloLetras {
+ ValidadorNombreSoloLetras(v : ValidadorEstudiante)
+ validar(id, nombre, edad) : Resultado
}
}

```

```

' =====
' ADAPTER
' =====

```

```

package "adapter" {
class LectorEstudiantesExternos {
+ leerDesdeCsv(ruta : String) : List<Estudiante>
+ leerDesdeJson(ruta : String) : List<Estudiante>
}

```

```

interface AdaptadorEstudiante <<interface>> {
+ adaptar() : Estudiante
}

```

```

class AdaptadorCsvEstudiante {
- estudianteCsv : EstudianteCsv
--
+ adaptar() : Estudiante
}

```

```

class AdaptadorJsonEstudiante {
- estudianteJson : EstudianteJson
--
+ adaptar() : Estudiante
}

```

```

class EstudianteCsv {
- codigo : String
- nombreCompleto : String
- anios : int

```

```

--
+ getCodigo() : String
+ getNombreCompleto() : String
+ getAnios() : int
}

class EstudianteJson {
- studentId : String
- name : String
- age : int
--
+ getStudentId() : String
+ getName() : String
+ getAge() : int
}
}

' =====
' REPOSITORY
' =====

package "repository" {
class RepositorioEstudiante {
- lista : List<Estudiante>
--
+ existId(id : String) : boolean
+ guardar(e : Estudiante)
+ buscarPorId(id : String) : Estudiante
+ actualizar(e : Estudiante)
+ eliminar(id : String)
+ listarTodos() : List<Estudiante>
}
}

' =====
' MODEL
' =====

package "model" {
class Estudiante {
- id : String
- nombre : String
- edad : int
--
+ getId() : String
+ getNombre() : String
+ getEdad() : int
+ setId(id : String)
+ setNombre(nombre : String)
+ setEdad(edad : int)

```

```
}  
}
```

```
'  
=====
```

' RELACIONES — VIEW → CONTROLLER

```
'  
=====
```

FormularioCrudEstudiante -down-> ControlEstudiante : usa

```
'  
=====
```

' RELACIONES — CONTROLLER

```
'  
=====
```

ControlEstudiante -down-> Resultado : retorna
ControlEstudiante -right-> RepositorioEstudiante : usa
ControlEstudiante -left-> ValidadorEstudiante : usa
ControlEstudiante -down-> LectorEstudiantesExternos : usa

```
'  
=====
```

' RELACIONES — DECORATOR

```
'  
=====
```

ValidadorBasico ..|> ValidadorEstudiante
ValidadorDecorador ..|> ValidadorEstudiante
ValidadorDecorador o--> ValidadorEstudiante : wraps
ValidadorEdadMaxima --|> ValidadorDecorador
ValidadorNombreSoloLetras --|> ValidadorDecorador

```
'  
=====
```

' RELACIONES — ADAPTER

```
'  
=====
```

LectorEstudiantesExternos -down-> AdaptadorCsvEstudiante : instancia
LectorEstudiantesExternos -down-> AdaptadorJsonEstudiante : instancia
AdaptadorCsvEstudiante ..|> AdaptadorEstudiante
AdaptadorJsonEstudiante ..|> AdaptadorEstudiante
AdaptadorCsvEstudiante -down-> EstudianteCsv : usa
AdaptadorJsonEstudiante -down-> EstudianteJson : usa
AdaptadorCsvEstudiante ..> Estudiante : <<crea>>
AdaptadorJsonEstudiante ..> Estudiante : <<crea>>

```
'  
=====
```

' RELACIONES — REPOSITORY → MODEL

```
'  
=====
```

RepositorioEstudiante -right-> Estudiante : almacena

@enduml

Matriz de trazabilidad entre requisitos, componentes y evidencias

ID	Requisito funcional	Operación	Componente principal	Evidencia requerida
RF-AD-01	El sistema debe importar datos desde estructuras CSV incompatibles (codigo, nombreCompleto, anios).	adaptar()	AdaptadorCsvEstudiante, LectorEstudiantesExternos	Consistencia en la tabla del CRUD tras procesar un archivo CSV de ejemplo.
RF-AD-02	El sistema debe importar datos desde esquemas JSON externos (studentId, name, age).	adaptar()	AdaptadorJsonEstudiante, LectorEstudiantesExternos	Inserción exitosa de registros mapeados desde el formato en inglés hacia el backend.
RF-DEC-01	El sistema debe validar que la edad ingresada no sea superior a un límite máximo parametrizado sin modificar el código base.	validar()	ValidadorEdadMaxima, ValidadorDecorador	Alerta visual o excepción capturada al ingresar un estudiante con edad inválida (ej. 150).
RF-DEC-02	El sistema debe rechazar nombres que contengan caracteres numéricos o símbolos especiales de	validar()	ValidadorNombreSoloLetras, ValidadorDecorador	Bloqueo de la transacción de guardado ante nombres inválidos como "Carlos 123".

	forma modular.			
--	----------------	--	--	--

Validaciones y restricciones del sistema

Restricción / validación	Ubicación	Descripción
Traducción de campos CSV	AdaptadorCsvEstudiante.adaptar()	Convierte el objeto DTO EstudianteCsv en una entidad limpia Estudiante reasignando las propiedades asimétricas.
Mapeo de Lenguaje JSON	AdaptadorJsonEstudiante.adaptar()	Traduce del inglés al español las propiedades e instancia un objeto de dominio válido.
Frontera de Edad Máxima	ValidadorEdadMaxima.validar()	Verifica si estudiante.getEdad() > edadMaxima. Si se incumple, interrumpe el flujo y retorna un Resultado fallido.
Restricción de Solo Letras	ValidadorNombreSoloLetras.validar()	Aplica una expresión regular <code>^[a-zA-ZáéíóúÁÉÍÓÚñÑ]+\$</code> . Si el nombre tiene números, deniega el almacenamiento.

Evidencias de ejecución



CRUD Estudiantes - U2T2 Adapter y Decorator



ID:

Nombre:

Edad:

Agregar

Actualizar

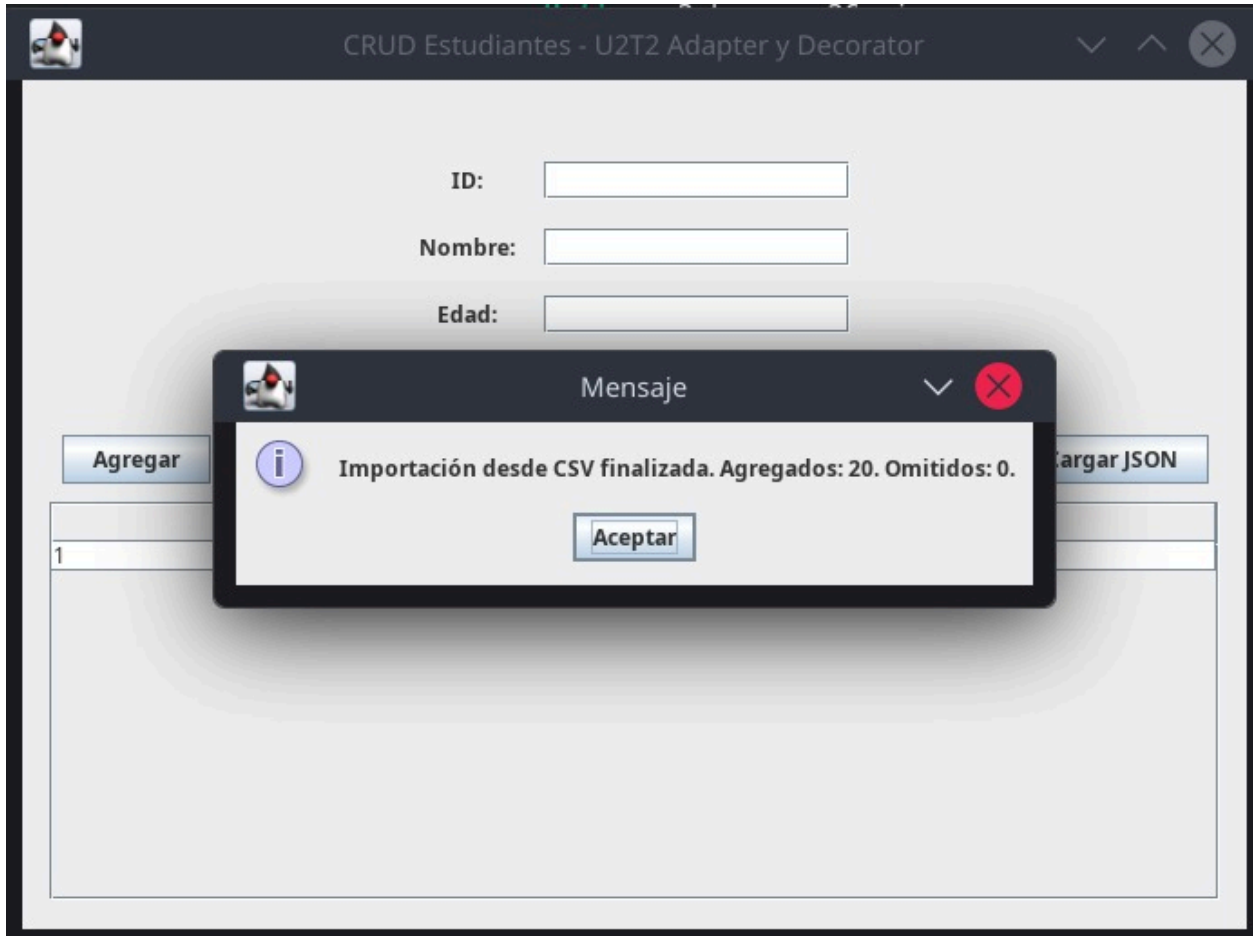
Eliminar

Mostrar Todo

Cargar CSV

Cargar JSON

ID	Nombre	Edad
1	Stiven Diaz	21





CRUD Estudiantes - U2T2 Adapter y Decorator



ID:

Nombre:

Edad:

Agregar

Actualizar

Eliminar

Mostrar Todo

Cargar CSV

Cargar JSON

ID	Nombre	Edad
1	Stiven Diaz	21

CRUD Estudiantes - U2T2 Adapter y Decorator

ID:

Nombre:

Edad:

Agregar

Cargar JSON

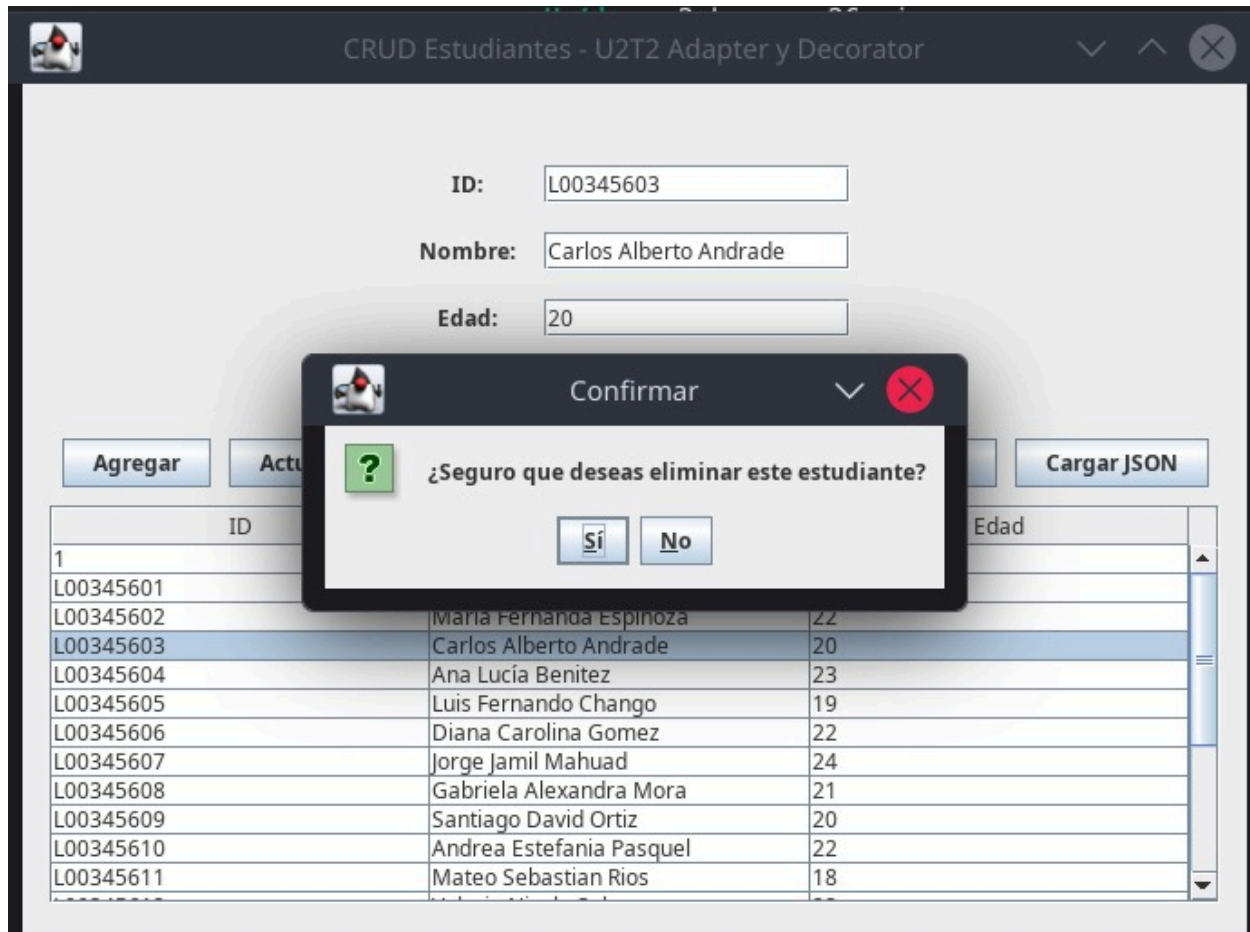
1		
L00345601		
L00345602	Maria Fernanda Espinoza	22
L00345603	Carlos Alberto Andrade	20
L00345604	Ana Lucia Benitez	23
L00345605	Luis Fernando Chango	19
L00345606	Diana Carolina Gomez	22
L00345607	Jorge Jamil Mahuad	24
L00345608	Gabriela Alexandra Mora	21
L00345609	Santiago David Ortiz	20
L00345610	Andrea Estefania Pasquel	22
L00345611	Mateo Sebastian Rios	18

Mensaje

i

Importación desde JSON finalizada. Agregados: 0. Omitidos: 20.

Aceptar



CONCLUSIONES

- El patrón Adapter eliminó el acoplamiento rígido entre las fuentes de información externas y el núcleo del software, permitiendo que el controlador central trabaje siempre de forma agnóstica con entidades del dominio local.
- El patrón Decorator demostró la factibilidad de inyectar dinámicamente lógicas y políticas de negocio adicionales sobre un servicio operativo sin alterar una sola línea de código de ValidadorBasico, respetando de forma estricta el principio Abierto/Cerrado.
- La cohesión arquitectónica se mantiene elevada debido a que los adaptadores se localizan exclusivamente en su paquete de integración y los decoradores actúan como filtros apilables previos a la capa de datos.

RECOMENDACIONES

- Aislar por completo las dependencias de librerías externas de parsing (como Gson o Jackson) dentro de las implementaciones concretas de los adaptadores para no contaminar otras capas.

- Utilizar fábricas simples (Factory Pattern) en iteraciones posteriores para instanciar las cadenas de decoradores, evitando código de configuración manual extenso en el controlador de la interfaz visual.

PARTE 3: U2T3. Aplicación de Observer y Strategy en el CRUD del estudiante

Descripción del problema

A medida que la aplicación escala a su versión final (crudEstudianteT3), los requisitos funcionales de dinamismo y respuesta ante eventos s vuelven complejos:

Notificación Desacoplada de Operaciones (Problema de Eventos): Se requiere que cada vez que ocurra una transacción de escritura en el CRUD (crear, actualizar, eliminar), múltiples módulos periféricos del sistema reaccionen instantáneamente (por ejemplo, imprimiendo un log inmediato en consola, registrando una bitácora detallada de auditoría de seguridad para los administradores o manteniendo un historial cronológico de estados). Implementar esto llamando directamente a cada módulo desde el controlador generaría un acoplamiento inaceptable y violaría el principio de Responsabilidad Única (SRP).

Algoritmia e Intercambio Dinámico (Problema de Comportamiento Variable): El usuario en la interfaz gráfica debe tener la libertad de alternar en tiempo real los criterios de búsqueda (búsqueda exacta por ID frente a búsqueda parcial e inductiva por Nombre) y las estrategias de visualización y ordenamiento de la tabla (ordenar alfabéticamente por ID frente a ordenar numéricamente por la edad). Hardcodear estas lógicas con condicionales anidados (if/else masivos) volvería al controlador rígido y propenso a errores ante nuevos algoritmos.

La solución se consolida aplicando dos patrones de diseño comportamentales. El patrón Observer establece una infraestructura reactiva donde el controlador publica eventos operacionales y los observadores registrados reaccionan de forma independiente. El patrón Strategy encapsula las familias de algoritmos de búsqueda y ordenamiento en componentes intercambiables y parametrizables, permitiendo su mutación en caliente según las acciones del usuario.

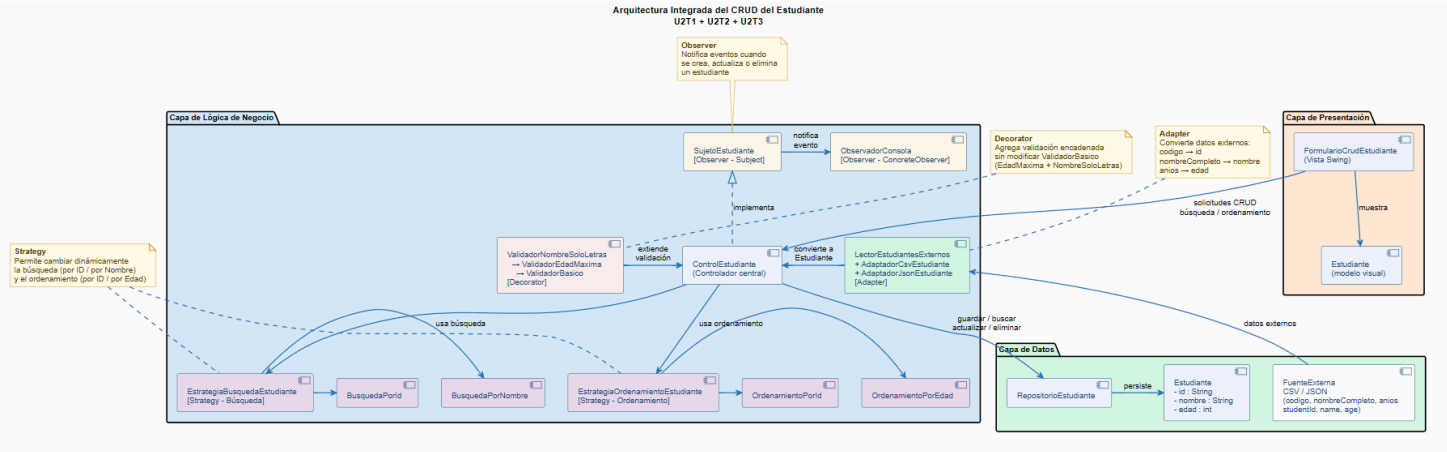
Elemento de la arquitectura y clases del proyecto

Capa / elemento	Clase en el proyecto	Responsabilidad
Eventos / observer	SujetoEstudiante	Interfaz estructural que define los métodos de gestión de suscripciones: agregarObservador(), eliminarObservador() y notificarObservadores().

Eventos observer	/	ObservadorEstudiante	Interfaz contractual que unifica a los suscriptores mediante la operación reactiva actualizar(String evento, Estudiante e).
Eventos observer	/	ObservadorConsola	Observador concreto que realiza impresiones formateadas e inmediatas del flujo transaccional en la terminal de desarrollo.
Eventos observer	/	ObservadorAuditoria	Observador concreto encargado de estructurar y persistir registros detallados de seguridad sobre operaciones atómicas del CRUD.
Eventos observer	/	ObservadorHistorial	Observador concreto que indexa mapas de estado cronológicos de la entidad para el seguimiento de cambios en el tiempo.
Estrategias strategy	/	EstrategiaBusquedaEstudiante	Interfaz que abstrae la familia de algoritmos de filtrado con el método buscar(List<Estudiante> lista, String criterio).
Estrategias strategy	/	BusquedaPorId	Estrategia concreta que ejecuta filtrados atómicos exactos comparando cadenas binarias directas.
Estrategias strategy	/	BusquedaPorNombre	Estrategia concreta que realiza búsquedas predictivas por subcadenas parciales empleando el método contains().
Estrategias strategy	/	EstrategiaOrdenamientoEstudiante	Interfaz que define el contrato de organización formal a través del método

			ordenar(List<Estudiante> lista).
Estrategias strategy	/	OrdenamientoPorId	Estrategia concreta que efectúa la ordenación ascendente utilizando un orden lexicográfico.
Estrategias strategy	/	OrdenamientoPorEdad	Estrategia concreta que realiza la clasificación numérica ascendente a través de comparadores enteros.
Controlador controller	/	ControlEstudiante	Actúa como el gran orquestador. Implementa SujetoEstudiante, despacha los eventos a los observadores y posee las referencias inyectables hacia las estrategias activas.

Modelo de arquitectura en PlantUML



@startuml U2T3_Arquitectura_Integrada

skinparam backgroundColor #FEFEFE
skinparam defaultFontName Arial
skinparam defaultFontSize 12
skinparam ArrowColor #2E75B6
skinparam ArrowThickness 1.5

skinparam rectangle {
 BackgroundColor #FFFFFF
 BorderColor #2E75B6
 FontColor #1F4E79
 BorderThickness 1.5
}

skinparam component {
 BackgroundColor #EBF3FB
 BorderColor #1F4E79
 FontColor #1F4E79
}

skinparam note {
 BackgroundColor #FFFBE6
 BorderColor #C8860A
 FontColor #5C3D00
}

skinparam package {
 BorderThickness 2
}

title Arquitectura Integrada del CRUD del Estudiante\nU2T1 + U2T2 + U2T3

' _____
' NOTAS DE PATRONES (arriba)
' _____

note as N_STRATEGY
 Strategy
 Permite cambiar dinámicamente
 la búsqueda (por ID / por Nombre)
 y el ordenamiento (por ID / por Edad)
end note

note as N_OBSERVER
 Observer
 Notifica eventos cuando
 se crea, actualiza o elimina
 un estudiante
end note

note as N_DECORATOR

```
<b>Decorator</b>
Agrega validación encadenada
sin modificar ValidadorBasico
(EdadMaxima + NombreSoloLetras)
end note
```

```
note as N_ADAPTER
<b>Adapter</b>
Convierte datos externos:
codigo → id
nombreCompleto → nombre
anios → edad
end note
```

```
' _____
'  CAPA DE PRESENTACIÓN
' _____
```

```
package "Capa de Presentación" #FFE8D6 {
  component "FormularioCrudEstudiante\n(Vista Swing)" as VISTA
  component "Estudiante\n(modelo visual)" as MOD_V
```

```
  VISTA -down-> MOD_V : muestra
}
```

```
' _____
'  CAPA DE LÓGICA DE NEGOCIO
' _____
```

```
package "Capa de Lógica de Negocio" #D6EAF8 {
```

```
  ' -- Adapter --
  component "LectorEstudiantesExternos\n+ AdaptadorCsvEstudiante\n+
  AdaptadorJsonEstudiante\n[Adapter]" as ADAPTER #D5F5E3
```

```
  ' -- Decorator --
  component "ValidadorNombreSoloLetras\n → ValidadorEdadMaxima\n →
  ValidadorBasico\n[Decorator]" as DECORATOR #FDEDEC
```

```
  ' -- Controlador central --
  component "ControlEstudiante\n(Controlador central)" as CTRL
```

```
  ' -- Observer --
  component "SujetoEstudiante\n[Observer - Subject]" as SUJETO #FEF9E7
```

```
  component "ObservadorConsola\n[Observer - ConcreteObserver]" as OBS_CONSOLA
  #FEF9E7
```

```
  ' -- Strategy Búsqueda --
  component "EstrategiaBusquedaEstudiante\n[Strategy - Búsqueda]" as STRAT_B
```

```

#E8DAEF
component "BusquedaPorId" as B_ID #E8DAEF
component "BusquedaPorNombre" as B_NOM #E8DAEF

' -- Strategy Ordenamiento --
component "EstrategiaOrdenamientoEstudiante\n[Strategy - Ordenamiento]" as
STRAT_O #E8DAEF
component "OrdenamientoPorId" as O_ID #E8DAEF
component "OrdenamientoPorEdad" as O_EDAD #E8DAEF

' relaciones internas negocio
ADAPTER    -right-> CTRL    : convierte a\nEstudiante
DECORATOR  -right-> CTRL    : extiende\nvalidación
CTRL       .up.|> SUJETO    : implementa
SUJETO     -right-> OBS_CONSOLA : notifica\nevento

CTRL       -down-> STRAT_B   : usa búsqueda
STRAT_B    -right-> B_ID
STRAT_B    -right-> B_NOM

CTRL       -down-> STRAT_O   : usa ordenamiento
STRAT_O    -right-> O_ID
STRAT_O    -right-> O_EDAD
}

' _____
' CAPA DE DATOS
' _____

package "Capa de Datos" #D5F5E3 {
  component "RepositorioEstudiante" as REPO
  component "Estudiante\n- id : String\n- nombre : String\n- edad : int" as ENTITY

  component "FuenteExterna\nCSV / JSON\n(codigo, nombreCompleto,
  anios\nstudentId, name, age)" as FUENTE #FD FEFE

  REPO -right-> ENTITY : persiste
}

' _____
' RELACIONES ENTRE CAPAS
' _____

VISTA      -down-> CTRL    : solicitudes CRUD\nbúsqueda / ordenamiento
CTRL       -down-> REPO    : guardar / buscar\nactualizar / eliminar
FUENTE     -up-> ADAPTER   : datos externos

' _____
' NOTAS → COMPONENTES
' _____

```

```

N_STRATEGY .. STRAT_B
N_STRATEGY .. STRAT_O
N_OBSERVER .. SUJETO
N_DECORATOR .. DECORATOR
N_ADAPTER .. ADAPTER

```

@enduml

Matriz de trazabilidad entre requisitos, componentes y evidencias

ID	Requisito funcional	Operación	Componente principal	Evidencia requerida
RF-OBS-01	Notificar automáticamente a los sistemas de monitoreo cuando se agrega un estudiante.	Evento: "Estudiante agregado"	ControlEstudiante, ObservadorConsola	Impresión en terminal del log formateado con los datos del nuevo estudiante.
RF-OBS-02	Registrar logs detallados y alertas en la bitácora al modificar datos del estudiante.	Evento: "Estudiante actualizado"	ControlEstudiante, ObservadorAuditoria	Entrada persistida en el componente de auditoría con el estado post-modificación.
RF-OBS-03	Alertar a los observadores e historiales al remover un registro del sistema.	Evento: "Estudiante eliminado"	ControlEstudiante, ObservadorHistorial	Captura de pantalla de la consola reflejando la salida reactiva tras pulsar "Eliminar".
RF-STR-01	Proveer un mecanismo de búsqueda exacta utilizando el	Algoritmo: Búsqueda PorId	BusquedaPorId, ControlEstudiante	Filtrado instantáneo en la tabla de la interfaz al ingresar un ID idéntico.

	identificador (ID).			
RF-STR-02	Proveer búsquedas parciales predictivas a través del atributo Nombres.	Algoritmo: Búsqueda PorNombre	BusquedaPorNombre, ControlEstudiante	Despliegue de coincidencias usando subcadenas (ej. escribir "Carlos" y listar coincidencias).
RF-STR-03	Ordenar los registros visuales de la tabla alfabéticamente por su clave primaria.	Algoritmo: OrdenamientoPorId	OrdenamientoPorId, ControlEstudiante	Reorganización inmediata de la grilla de la UI en orden lexicográfico.
RF-STR-04	Clasificar la grilla de estudiantes numéricamente de forma ascendente según su edad.	Algoritmo: OrdenamientoPorEdad	OrdenamientoPorEdad, ControlEstudiante	Reordenación de filas de menor a mayor edad al cambiar el selector de la vista.
ID	Requisito funcional / técnico	Evento / Estrategia Activa	Componente principal	Evidencia requerida
RF-OBS-01	Notificar automáticamente a los sistemas de monitoreo cuando se agrega un estudiante.	Evento: "Estudiante agregado"	ControlEstudiante, ObservadorConsola	Impresión en terminal del log formateado con los datos del nuevo estudiante.

RF-OBS-02	Registrar logs detallados y alertas en la bitácora al modificar datos del estudiante.	Evento: "Estudiante actualizado"	ControlEstudiante, ObservadorAuditoria	Entrada persistida en el componente de auditoría con el estado post-modificación.
-----------	---	----------------------------------	--	---

Validaciones y restricciones del sistema

Restricción / validación	Ubicación	Descripción
Desacoplamiento de Eventos	ControlEstudiante.notificarObservadores()	Recorre la colección de suscriptores y ejecuta .actualizar() de forma polimórfica. El controlador jamás conoce las clases concretas de los observadores.
Filtro Parcial Inductivo	BusquedaPorNombre.buscar()	Aplica el método .contains() convirtiendo los textos a minúsculas (.toLowerCase()) para asegurar que la búsqueda ignore diferencias de caja.
Invariabilidad del Repositorio	OrdenamientoPorEdadPorId.ordenar()	Trabaja obligatoriamente sobre una copia superficial de la lista original (new ArrayList<>(lista)). Esto restringe que la ordenación visual altere la persistencia real del repositorio.
Intercambio Dinámico	Métodos Setter en ControlEstudiante	Los métodos setEstrategiaBusqueda() y setEstrategiaOrdenamiento() permiten reasignar las

(Mutación)		referencias de tipo interfaz en caliente según la acción del usuario.
Aislamiento Arquitectónico	Paquete com.espe.edu.strategy / observer	Las lógicas algorítmicas y reactivas quedan confinadas en la capa de negocio. La vista Swing se limita a invocar los Setters del controlador sin computar procesos.

Evidencias de ejecución

CRUD Estudiantes - U2T2 Adapter y Decorator

ID:

Nombre:

Edad:

ID	Nombre	Edad
123	Lucas	23
321	Xavier	23
456	Victor	23

Criterio:

CRUD Estudiantes - U2T2 Adapter y Decorator

ID:

Nombre:

Edad:

ID	Nombre	Edad
123	Lucas	23
321	Xavier	23
456	Victor	23

Message

Estudiante actualizado

Criterio:

CRUD Estudiantes - U2T2 Adapter y Decorator

ID:

Nombre:

Edad:

ID	Nombre	Edad
123	Lucas	22
321	Xavier	23
456	Victor	23

Message

Estudiante eliminado

Criterio:

CRUD Estudiantes - U2T2 Adapter y Decorator

ID:

Nombre:

Edad:

ID	Nombre	Edad
123	Lucas	22
456	Victor	23
L00345601	Juan Carlos Perez	21
L00345602	Maria Fernanda Espinoza	22
L00345603	Carlos Alberto Andrade	20
L00345604	Ana Lucia Benitez	23
L00345605	Luis Fernando Chango	19
L00345606	Diana Carolina Gomez	22
L00345607	Jorge Jamil Mahuad	24
L00345608	Gabriela Alexandra Mora	21
L00345609	Santiago David Ortiz	20
L00345610	Andrea Estefania Pasquel	22

Criterio:

CRUD Estudiantes - U2T2 Adapter y Decorator

ID:

Nombre:

Edad:

ID	Nombre	Edad
L00345611	Mateo Sebastian Rios	18
L00345605	Luis Fernando Chango	19
L00345617	Daniel Ricardo Lopez	19
L00345603	Carlos Alberto Andrade	20
L00345609	Santiago David Ortiz	20
L00345614	Camila Belen Villacis	20
L00345601	Juan Carlos Perez	21
L00345608	Gabriela Alexandra Mora	21
L00345613	Kevin Alejandro Torres	21
L00345618	Michelle Dayana Flores	21
123	Lucas	22
L00345602	Maria Fernanda Espinoza	22

Criterio:

CONCLUSIONES

- La integración de los patrones Observer y Strategy en el entregable final dota al CRUD de estudiantes de una flexibilidad operativa superior, permitiendo que la interfaz Swing responda reactivamente a cambios de estado y permute algoritmos en caliente de forma transparente.

- El uso de copias de las colecciones dentro de las estrategias de ordenación garantiza la consistencia de los datos en el repositorio, demostrando buenas prácticas en el manejo de estructuras en memoria.
- La arquitectura consolidada cumple con los cinco principios SOLID, sentando bases sólidas para transiciones de infraestructura complejas sin incurrir en deudas técnicas significativas.

RECOMENDACIONES

- Se recomienda complementar la capa de datos (RepositorioEstudiante) utilizando el patrón DAO (Data Access Object) para realizar una migración fluida de la lista ArrayList temporal hacia una base de datos relacional robusta (como MySQL o PostgreSQL) sin alterar el controlador.
- Se aconseja incorporar en el catálogo de estrategias algoritmos de filtrado multicriterio complejos como BusquedaPorRangoDeEdad o validaciones avanzadas mediante expresiones regulares sin modificar la vista o la lógica de control.
- Canales de Alerta Heterogéneos: Diseñar observadores concretos adicionales tales como ObservadorMail o ObservadorNotificacionPush para simular la integración con microservicios de mensajería externos en un entorno real distribuido.
- Sustituir los flujos e impresiones de pruebas manuales por un entorno de integración continua formalizado utilizando frameworks estándares de la industria como JUnit 5 y Mockito.

REFERENCIAS

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Kendall, K. E., & Kendall, J. E. (2011). *Análisis y設計 de sistemas* (8.ª ed.). Pearson Educación.
- Pressman, R. S. (2010). *Ingeniería del software: Un enfoque práctico* (7.ª ed.). McGraw-Hill.
- Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.